



ASCII

# Pengantar Pemrograman Berorientasi Objek, Class, dan Object



## Pendahuluan

Pada awalnya, pengembangan sebuah program biasanya tidak terlalu sulit karena kebutuhan yang harus ditangani masih terbatas. Data yang digunakan juga masih sedikit sehingga programmer dapat menyimpan dan mengelolanya dengan cara yang sederhana. Namun, seiring bertambahnya kebutuhan program, jumlah data yang harus disimpan dan diproses juga semakin banyak. Kondisi tersebut dapat membuat kode program menjadi lebih panjang dan mulai sulit dikelola.

Sebagai contoh, sebuah program yang digunakan untuk mengelola produk dapat menyimpan beberapa informasi seperti nama, harga, stok, kategori, merek, dan informasi lainnya. Jika program hanya menangani satu produk, penyimpanan data masih terlihat sederhana:

```
product_name = "Keyboard"
product_price = 500000
product_stock = 10
product_category = "Aksesoris"
product_brand = "Logitech"
```

Ketika program perlu mengelola produk lainnya, programmer perlu membuat kumpulan data yang sama kembali untuk produk tersebut.

```
product1_name = "Keyboard"
product1_price = 500000
product1_stock = 10
product1_category = "Aksesoris"
product1_brand = "Logitech"

product2_name = "Mouse"
product2_price = 300000
product2_stock = 15
product2_category = "Aksesoris"
product2_brand = "Logitech"
```



Cara tersebut memang dapat digunakan, tetapi mulai terlihat bahwa semakin banyak produk yang dikelola, semakin banyak pula data yang harus ditulis dan dipelihara. Jika jumlah produk mencapai puluhan atau ratusan, penulisan seperti ini akan membuat kode menjadi panjang dan sulit dikelola. Data yang sebenarnya memiliki struktur yang sama juga harus dituliskan secara berulang.

Salah satu cara yang umum digunakan untuk mengatasi permasalahan tersebut adalah dengan mengelompokkan data menggunakan list yang berisi beberapa dictionary. Dengan cara ini, setiap produk dapat disimpan sebagai satu kesatuan.

```
products = [  
    {  
        "name": "Keyboard",  
        "price": 500000,  
        "stock": 10,  
        "category": "Aksesoris",  
        "brand": "Logitech"  
    },  
    {  
        "name": "Mouse",  
        "price": 300000,  
        "stock": 15,  
        "category": "Aksesoris",  
        "brand": "Logitech"  
    }  
]
```

Struktur tersebut lebih rapi karena data setiap produk telah dikelompokkan. Programmer juga dapat menambahkan produk baru tanpa harus membuat banyak variabel secara terpisah.

Namun, penggunaan list dan dictionary tetap memiliki keterbatasan. Data di dalam dictionary diakses berdasarkan nama *key* yang dituliskan secara manual. Sebagai contoh, untuk mengambil harga sebuah produk, programmer harus menuliskan:



```
products[0]["price"]
```

Begitu pula ketika ingin mengubah stok:

```
products[0]["stock"] = 20
```

Programmer harus **mengingat** nama **key** yang digunakan. Kesalahan penulisan seperti berikut:

```
products[0]["stok"]
```

dapat menyebabkan kesalahan karena *key* yang sebenarnya adalah:

```
"stock"
```

Permasalahan tidak hanya berkaitan dengan penyimpanan data. Ketika kebutuhan program bertambah, produk juga membutuhkan berbagai proses untuk mengelola datanya. Misalnya, program perlu menyediakan fungsi untuk menambah stok, mengurangi stok, mengubah harga, dan menghitung diskon.

```
def tambah_stok(product, jumlah):  
    product["stock"] += jumlah  
  
def kurangi_stok(product, jumlah):  
    if product["stock"] ≥ jumlah:  
        product["stock"] -= jumlah  
  
def ubah_harga(product, harga_baru):  
    product["price"] = harga_baru
```

Pada contoh tersebut, data produk berada di dalam dictionary, sedangkan fungsi yang digunakan untuk mengelola data tersebut berada di luar dictionary. Ketika program semakin besar, jumlah data dan fungsi juga dapat terus bertambah. Akibatnya,



programmer harus mengelola banyak bagian program yang saling berkaitan tetapi berada di tempat yang **berbeda**.

Semakin banyak data dan proses yang ditambahkan, semakin penting bagi programmer untuk memiliki cara yang lebih terstruktur dalam mengorganisasi program. Data yang berkaitan dengan suatu bagian program dapat dikelompokkan bersama dengan fungsi yang digunakan untuk mengelola data tersebut.

Pendekatan yang dapat digunakan untuk mengatasi kebutuhan tersebut adalah **Object-Oriented Programming (OOP)** atau **Pemrograman Berorientasi Objek (PBO)**. OOP memungkinkan programmer mengelompokkan **data dan perilaku yang saling berkaitan** ke dalam sebuah objek.

Sebagai contoh, data dan fungsi yang sebelumnya terpisah dapat digabungkan ke dalam sebuah *class*:

```
class Product:
    def __init__(self, name, price, stock, category, brand):
        self.name = name
        self.price = price
        self.stock = stock
        self.category = category
        self.brand = brand

    def tambah_stok(self, jumlah):
        self.stock += jumlah

    def kurangi_stok(self, jumlah):
        if self.stock >= jumlah:
            self.stock -= jumlah

    def ubah_harga(self, harga_baru):
        self.price = harga_baru
```

Dari *class* tersebut, programmer dapat membuat beberapa objek produk tanpa harus menuliskan struktur data dan fungsi yang sama secara berulang. Data dan fungsi yang



berkaitan dengan produk juga berada dalam satu struktur yang jelas, sehingga programmer dapat mengelola, memahami, dan mengembangkan kode dengan lebih mudah.

```
mouse = Product("Mouse", 300000, 15, "Aksesoris", "Logitech")

mmouse.tambah_stok(5)
mouse.kurangi_stok(2)
mouse.ubah_harga(280000)
```

Pada contoh tersebut, mouse merupakan objek yang dibuat dari class Product. Objek mouse memiliki data seperti name, price, stock, category, brand, weight, color, rating, discount, dan status yang disebut sebagai **attribute**. Sedangkan fungsi seperti tambah\_stok(), kurangi\_stok(), ubah\_harga(), dan hitung\_diskon() disebut sebagai **method**.

OOP tidak hanya digunakan untuk menyimpan data, tetapi juga untuk **mengorganisasi data** dan **proses** yang berkaitan dengannya ke dalam struktur yang lebih jelas. Pendekatan ini menjadi semakin berguna ketika program memiliki banyak data, fitur, dan proses yang saling berhubungan.

## Pemrograman Berorientasi Objek (PBO)

OOP (Object-Oriented Programming) atau yang dalam bahasa Indonesia disebut Pemrograman Berorientasi Objek (PBO) merupakan **paradigma** pemrograman yang menggunakan **objek** sebagai dasar dalam perancangan dan pengembangan program. Pemrograman Berorientasi Objek (PBO) menjadikan objek sebagai bagian utama dalam program. Setiap objek memiliki **data (attribute)** dan **fungsi (method)**. PBO memiliki empat prinsip utama, yaitu *Encapsulation*, *Abstraction*, *Inheritance*, dan *Polymorphism*. Keempat prinsip tersebut menjadi dasar dalam penerapan Pemrograman Berorientasi Objek dan akan dibahas lebih lanjut dalam bab-bab selanjutnya. Kelebihan dari PBO adalah sebagai berikut.



- OOP membuat program lebih mudah dirawat karena struktur kode menjadi lebih terorganisir sehingga lebih mudah dibaca dan dipahami.
- Program lebih mudah disesuaikan ketika terdapat kebutuhan untuk menambah atau menghapus fitur maupun objek tertentu.

## Class

**Class** adalah rancangan atau pola dasar (seperti blueprint atau cetakan) yang digunakan untuk membuat objek. Di dalamnya, Class mendefinisikan ciri-ciri (**attribute**) dan perilaku (**method**) yang akan dimiliki oleh objek. Objek adalah hasil jadi dari sebuah class.

Analoginya seperti ini:

1. Class bisa diibaratkan sebagai cetakan kue atau desain laptop, yaitu gambaran umum tentang sebuah benda.
2. Objek adalah kue yang sudah jadi dari cetakan itu, atau laptop milik Budi, yang merupakan hasil nyata dari desain tersebut.

Dalam bahasa Python, penulisan `class` diawali dengan keyword `class`, kemudian diikuti dengan nama dari `class` tersebut. Nama `class` tidak boleh diawali dengan angka dan tidak dapat mengandung spasi. Penamaan `class` biasanya menggunakan **PascalCase**, yaitu setiap kata diawali dengan huruf kapital, termasuk kata pertama.

Berikut adalah contoh penulisan `class` dalam bahasa Python:

```
class Laptop:
    # isi dari class Laptop, berupa attribute dan method
    pass
```

Pada contoh tersebut, `class` merupakan keyword yang digunakan untuk mendefinisikan sebuah `class`, sedangkan `Laptop` merupakan nama `class`. Penggunaan **PascalCase** membuat nama `class` lebih mudah dibedakan dari variabel dan fungsi dalam program.



## Object

**Object** (biasa juga disebut sebagai **instance**) adalah hasil instansiasi dari sebuah class. Jika class dapat diibaratkan sebagai cetakan atau rancangan, maka object merupakan hasil nyata yang dibuat berdasarkan rancangan tersebut.

Masih menggunakan analogi Laptop, sebuah class Laptop dapat digunakan untuk membuat beberapa object, seperti laptop1, laptop2, dan laptop3. Setiap object dibuat berdasarkan class yang sama, tetapi dapat memiliki data atau nilai attribute yang berbeda.

Setiap object yang dibuat dari sebuah class memiliki attribute dan method yang telah didefinisikan oleh class tersebut. Dengan demikian, sebuah class dapat digunakan sebagai dasar untuk membuat banyak object dengan struktur yang sama.

Proses membuat object dari sebuah class disebut **instansiasi (instantiation)**. Berikut adalah contoh proses membuat object di Python:

```
laptop = Laptop()
```

## Attribute

**Attribute** (kadang juga disebut **property** atau **field**) adalah data atau karakteristik yang dimiliki oleh sebuah **class** atau **objek**. Attribute digunakan untuk menyimpan informasi yang berkaitan dengan objek. Jika menggunakan analogi *class* Laptop, attribute dapat berupa merek, warna, harga, ukuran layar, dan informasi lainnya.

Dalam Python, attribute dapat ditentukan di dalam *class*. Berikut contoh sederhana:

```
class Laptop:  
    brand = "Lenovo"  
    price = 8000000
```

Pada contoh tersebut, *brand* dan *price* merupakan *attribute* dari *class* Laptop. *Attribute* tersebut digunakan untuk menyimpan informasi yang berkaitan dengan laptop.



## Metode (method)

**Method** adalah fungsi yang didefinisikan di dalam sebuah class dan digunakan untuk menggambarkan perilaku atau tindakan yang dapat dilakukan oleh objek. Jika menggunakan analogi class Laptop, contoh method dapat berupa menyalakan laptop, mematikan laptop, atau menghitung harga setelah diskon.

Pada dasarnya, method merupakan fungsi yang berada di dalam class. Method dapat menerima parameter dan dapat mengembalikan nilai menggunakan return, sama seperti fungsi pada umumnya.

Berikut contoh class Laptop yang memiliki *attribute* dan method:

```
class Laptop:
    brand = "Lenovo"
    price = 8000000

    def change_price(self, new_price):
        self.price = new_price
```

Pada contoh `change_price()` merupakan method dari class Laptop.

## Constructor

**Constructor** adalah fungsi atau *method* yang dipanggil **secara otomatis** ketika sebuah objek dari *class* **dibuat**. Dalam Python, constructor dituliskan menggunakan metode `__init__()`.

Constructor berguna ketika sebuah *class* memiliki *attribute* yang nilainya perlu ditentukan saat object dibuat. Dengan constructor, programmer dapat memberikan nilai attribute secara langsung pada saat proses instansiasi.

Sebagai contoh, sebelumnya *attribute* pada *class* Laptop dituliskan langsung di dalam *class*.



```
class Laptop:
    brand = "Lenovo"
    price = 8000000

    def change_price(self, new_price):
        self.price = new_price
```

Cara tersebut membuat nilai *brand* dan *price* telah ditentukan di dalam *class*. Apabila programmer ingin membuat beberapa objek laptop dengan merek dan harga yang berbeda, nilai tersebut perlu ditentukan dengan cara lain.

Untuk memberikan nilai *attribute* yang dapat berbeda pada setiap object, *attribute* dapat dituliskan di dalam metode `__init__()`.

```
class Laptop:
    def __init__(self, brand, price):
        self.brand = brand
        self.price = price

    def change_price(self, new_price):
        self.price = new_price
```

Pada contoh tersebut, `__init__()` merupakan metode khusus yang digunakan untuk menginisialisasi object. Parameter *brand* dan *price* digunakan untuk menerima nilai yang diberikan ketika object dibuat.

Selanjutnya, programmer dapat membuat beberapa object dengan nilai *attribute* yang berbeda:

```
laptop1 = Laptop("Lenovo", 8000000)
laptop2 = Laptop("Asus", 10000000)
```

Object *laptop1* dan *laptop2* dibuat dari *class Laptop*, tetapi masing-masing memiliki nilai *attribute* yang berbeda.



Nilai tersebut dapat diakses melalui object:

```
print(laptop1.brand)
print(laptop1.price)

print(laptop2.brand)
print(laptop2.price)
```

Output:

```
Lenovo
8000000
Asus
10000000
```

Pada contoh tersebut, `self.brand` dan `self.price` merupakan attribute yang dimiliki oleh masing-masing object. Nilainya diperoleh dari parameter `brand` dan `price` yang diberikan ketika object dibuat.

## Keyword self

**self** adalah parameter yang digunakan dalam class Python untuk merujuk pada objek yang sedang digunakan. `self` memungkinkan sebuah method mengakses *attribute* dan *method* yang dimiliki oleh objek tersebut.

Perhatikan contoh berikut:

```
class Laptop:
    def __init__(self, brand, price):
        self.brand = brand
        self.price = price

    def change_price(self, new_price):
        self.price = new_price
```

Pada contoh tersebut, `self.brand` dan `self.price` merupakan attribute milik objek. Sementara itu, `self` pada method `change_price()` digunakan agar Python mengetahui attribute milik objek mana yang akan diakses atau diubah.



Dalam Python, `self` harus dituliskan sebagai parameter pertama pada method yang digunakan untuk bekerja dengan objek. Parameter tersebut secara otomatis menerima objek yang memanggil method.

Misalnya:

```
laptop = Laptop("Lenovo", 8000000)

laptop.change_price(7500000)
```

Ketika `change_price()` dipanggil melalui `laptop`, Python secara otomatis memberikan objek `laptop` kepada parameter pertama, yaitu `self`. Dengan demikian, pada:

```
self.price = new_price
```

`self.price` merujuk pada attribute `price` milik objek `laptop`.

Selain untuk mengakses attribute, `self` juga dapat digunakan untuk memanggil method lain yang berada dalam class yang sama. Untuk melakukannya, nama method dituliskan setelah `self` dan diikuti tanda kurung `()`.

Sebagai contoh:

```
class Laptop:
    def __init__(self, brand, price):
        self.brand = brand
        self.price = price

    def show_info(self):
        print(f"Brand: {self.brand}")
        print(f"Price: {self.price}")

    def change_price(self, new_price):
        self.price = new_price
        self.show_info()
```

Pada method `change_price()`, terdapat:



```
self.show_info()
```

Kode tersebut digunakan untuk memanggil method `show_info()` milik objek yang sedang digunakan. Ketika `change_price()` dipanggil oleh objek `laptop`, `self.show_info()` akan menjalankan `show_info()` pada objek `laptop` tersebut.

```
laptop = Laptop("Lenovo", 8000000)  
laptop.change_price(7500000)
```

## Output:

```
Brand: Lenovo  
Price: 7500000
```

Dengan demikian, `self` dapat digunakan untuk mengakses attribute maupun method milik objek yang sedang digunakan.

Secara sederhana, `self` dapat dipahami sebagai “objek ini”. Penggunaan `self` memungkinkan setiap objek mengakses dan mengelola attribute serta menjalankan method yang dimilikinya.